

Analyzing Performance of Multi-Threaded and Distributed Matrix Scaling Workloads on Cloud Infrastructure

Sri Venkat Bora
25164414
MSc in Cloud Computing

12th June 2026

1 Research Problem Background

The performance of coarse-grained, multi-threaded LU-factorisation code in MATLAB has been thoroughly studied on a variety of matrix sizes by Sabir and Alebrahim (2025) and they find that such a high-level, shared-memory LU factorisation algorithm rarely achieves more than a 2x speed-up for a 2x increase in number of cores, due to the fact that communication and synchronisation costs increase with the number of cores. They do not, however, explore the distributed, scale-out option – or how that will compare to the multi-threaded path – on cloud infrastructure, leaving that unexamined.

As discussed in Mochurad et al. (2026), parallel efficiency on a multi-core CPU decreases with the increasing number of cores, while in Torres Niño et al. (2025) the parallel performance of matrix-multiplication is shown to differ significantly across parallel back-ends even on a single node, suggesting that the performance is not only determined by the hardware but also by the parallel approach. This proposal compares the scalability of multi-threaded and distributed parallel processing of matrix workloads on AWS.

2 Research Question

How do multi-threaded and distributed parallel processing compare in completion time, memory footprint and core-distribution efficiency for matrix workloads of increasing size on AWS?

3 Justification

The distribution of a compute job is far more important than its raw dimension and determines whether a cloud team can realize a useful speed-up or end up paying for cores that don't do anything, but much of the time, the decision is made by rote rather than by measurement. This study will provide that evidence by comparing the single-machine multi-threaded matrix benchmark of Sabir and Alebrahim (2025) with the multi-threaded version of the same benchmark, but distributed across multiple machines on AWS. The configuration space is factored and multiplied for one general-purpose multi-core EC2 instance with either thread or process based execution and for a distributed system (Dask or Ray) using a small fixed cluster of two or three stopped instances after each execution. Only pure-Python threading is used in the global interpreter lock, not the native numerical kernels, to ensure a fair comparison of the library thread counts.

The time to complete the task, the peak memory consumption and core-distribution efficiency (which is the mean proportion of available vCPUs kept busy during the task) are measured by the operating-system counters and/or CloudWatch across a minimum of 5 iterations per size, and reported as mean, standard deviation and peak, and whether the distributed path beats the single-node path, but which may remain faster over the range of sizes tested. Only synthetic matrices are used, which are part of a small capped budget and do not involve the study of personal data and remain within the parameters of normal use.

4 Specific Items to Be Addressed

1. *Workload design*: design a set of synthetic dense matrix workloads, factorisation and multiplication, of increasing dimension, for example from 200 by 200 up to around 2000 by 2000.
2. *Parallel configurations*: study thread-based, process-based and distributed (Dask or Ray) Python implementations of the same workloads, the first two on one general-purpose multi-core EC2 instance and the distributed framework across a small fixed cluster of two or three EC2 instances.
3. *Completion time*: measure task completion time for each configuration as the matrix size grows, across at least five repeated runs summarised by mean and standard deviation.
4. *Memory and efficiency*: measure peak memory footprint and core-distribution efficiency, the mean proportion of available vCPUs kept usefully busy, for each configuration.
5. *Interpreter-lock effect*: examine how Python’s global interpreter lock constrains pure-Python threading but is bypassed by the native numerical kernels on these CPU-bound workloads.
6. *Guidance*: analyse the time, memory and efficiency trade-off and derive guidance on when multi-threading or a distributed framework suits small-to-medium cloud matrix workloads.

Bibliography

- Mochurad, L., Tsmots, I., Mostova, V. and Kystsiv, K. (2026) ‘Multi-level parallel CPU execution method for accelerated portion-based variant call format data processing’, *Computation*, 14(2), 48. doi: 10.3390/computation14020048.
- Sabir, O. and Alebrahim, R. (2025) ‘Coarse-grained column agglomeration parallel algorithm for LU factorization using multi-threaded MATLAB’, *Mathematics*, 13(2), 298. doi: 10.3390/math13020298.
- Torres Niño, L.A., Barrios Hernández, C.J. and Denneulin, Y. (2025) ‘Cross-platform performance evaluation of matrix multiplication: insights from MKL, cuBLAS, and SYCL’, *Computación y Sistemas*, 29(3). doi: 10.13053/cys-29-3-5567.